

ИНСТРУКЦИЯ ПО РАЗВЁРТЫВАНИЮ

Сервис прогнозирования инцидентов в инженерных коллекторах

Задача №8 «Город»

Конкурс «Лидеры цифровых трансформаций 2026»

Постановщик: Департамент жилищно-коммунального хозяйства города Москвы

Эксплуатант: АО «Москоллектор»

Команда «Дискреция творца»

Москва, сентябрь 2026

Содержание

1. Что потребуется	3
2. Запуск на своей машине	4
2.1. Вход	4
2.2. Порт занят	4
3. Развёртывание на сервере	5
3.1. Порядок действий	5
3.2. Что уезжает на сервер	5
3.3. Секреты	5
4. Доступ к серверному стенду	6
4.1. Туннель SSH — быстро и без настройки	6
4.2. Обратный прокси с TLS — для публичного адреса	6
4.3. Открыть порт напрямую	6
5. Эксплуатация	7
5.1. Резервная копия	7
5.2. Возврат к исходному набору вердиктов	7
5.3. Обновление версии	7
6. Настройки окружения	8
6.1. Переменные приложения	8
6.2. Переменные скрипта развёртывания	8
7. Если что-то пошло не так	9
7.1. Куда смотреть в первую очередь	9

1. Что требуется

Компонент	Требование
Docker	версия 24 и выше
Docker Compose	v2, обычно идёт плагином к Docker
Оперативная память	1 Гб на приложение и базу вместе
Место на диске	1 Гб под образ и том базы
Порт	8080, меняется одной переменной

Больше ничего не нужно. **Python, датасет и обученная модель для запуска стенда не требуются:** вердикты уезжают в образ компактным файлом инициализации, и приложение читает их из базы, а не считывает на месте.

2. Запуск на своей машине

```
git clone https://github.com/orbitalrailgun/lct2026-gorod-8.git concorde
cd concorde
docker compose up
```

Признак успешного старта — эти строки в выводе:

```
concorde-db-1 | database system is ready to accept connections
concorde-app-1 | PostgreSQL готов.
concorde-app-1 | схема создана
concorde-app-1 | назначено объектов: техник 2, группа реагирования 2
concorde-app-1 | загружено вердиктов: 162
concorde-app-1 | Запуск приложения на :8080
```

Интерфейс: **http://localhost:8080**

Проверка из командной строки:

```
curl http://localhost:8080/api/health
# {"status":"ok","database":true,"api_token_required":false}
```

2.1. Вход

Аутентификация двухфакторная, поэтому для проверяющего код показывается прямо в форме.

1. Выберите пользователя в списке.
2. Раскройте блок **«Код для демонстрации»** и нажмите **«Показать»**.
3. Введите шестизначный код и нажмите **«Войти»**.

Логин	Роль	Что доступно
ods	Диспетчер ОДС	вся сеть, настройки, журнал действий
dispatcher	Диспетчер района	вердикты района, решения, заявки
tehnik	Техник	просмотр по назначенным объектам
brigade	Группа реагирования	заявки и решения по своим объектам

Для полного обзора входите как `ods`.

2.2. Порт занят

Если 8080 уже кем-то занят:

```
APP_PORT=8081 docker compose up
```

3. Развёртывание на сервере

Для публичного стенда есть скрипт `deploy/remote.sh`. Он делает по SSH то же самое, что вы делали бы руками, и ничего сверх того.

3.1. Порядок действий

Шаг 1. Настройте вход по ключу. Не обязательно, но избавляет от ввода пароля:

```
CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh keys
```

Пароль спросят один раз — при копировании ключа.

Шаг 2. Проверьте машину.

```
CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh check
```

Скрипт убедится, что подключение есть, `docker` установлен и доступен текущему пользователю, и покажет свободное место. Если `docker` нет:

```
CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh install-docker
```

Шаг 3. Разверните.

```
CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh deploy
```

Первая сборка занимает несколько минут — ставятся зависимости. Повторный запуск обновляет стенд и данные не трогает.

3.2. Что уезжает на сервер

Около мегабайта: код, файлы инициализации, описание сборки.

Не уезжает: датасет, витрины, обученная модель, история `git`, документация и тесты. Поэтому серверу не нужны ни место под данные, ни память под обработку.

3.3. Секреты

При первом развёртывании на сервере создаётся файл `.env` со случайным паролем базы и случайным секретом сессий — сгенерированными там же. Через вашу историю команд они не проходят.

Файл исключён из синхронизации намеренно: иначе повторное развёртывание стирало бы его, пароль генерировался бы заново и переставал совпадать с тем, что зафиксирован в томе базы при первом запуске. Приложение тогда молча висит на «Ожидание PostgreSQL».

4. Доступ к серверному стенду

Стенд слушает только **127.0.0.1** и в сеть не выставлен. Это значение по умолчанию, и оно намеренное: страница входа показывает одноразовый код, поэтому открытый порт означает, что войти может любой, кто знает адрес.

4.1. Туннель SSH — быстро и без настройки

```
CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh tunnel
```

Команда держит туннель, пока вы её не прервёте. Интерфейс — на **http://localhost:8080**. Трафик идёт по зашифрованному каналу, на сервере ничего настраивать не нужно.

4.2. Обратный прокси с TLS — для публичного адреса

Штатный способ выставить стенд наружу на время экспертизы. Прокси работает на той же машине и ходит на петлю:

```
server {
    listen 443 ssl;
    server_name stand.example.ru;

    ssl_certificate      /etc/letsencrypt/live/stand.example.ru/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/stand.example.ru/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;

        # Интерфейс работает на веб-сокетах: без этих двух строк
        # страница загрузится, но останется пустой.
        proxy_set_header Upgrade      $http_upgrade;
        proxy_set_header Connection   "upgrade";

        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP     $remote_addr;
    }
}
```

4.3. Открыть порт напрямую

Можно, но осознанно:

```
CONCORDE_BIND=0.0.0.0 CONCORDE_HOST=user@203.0.113.10 deploy/remote.sh deploy
```

Скрипт предупредит об этом при каждом развёртывании. Не забудьте открыть порт в межсетевом экране: `ufw allow 8080/tcp`, `firewall-cmd --add-port=8080/tcp --permanent`, либо во входящих правилах облачной security group.

5. Эксплуатация

Задача	Локально	На сервере
Состояние	<code>docker compose ps</code>	<code>deploy/remote.sh status</code>
Журнал	<code>docker compose logs -f app</code>	<code>deploy/remote.sh logs 100</code>
Перезапуск	<code>docker compose restart app</code>	<code>deploy/remote.sh restart</code>
Остановка	<code>docker compose down</code>	<code>deploy/remote.sh down</code>
Адрес	—	<code>deploy/remote.sh url</code>

5.1. Резервная копия

```
docker compose exec db pg_dump -U concorde concorde > backup.sql
```

Восстановление:

```
docker compose exec -T db psql -U concorde concorde < backup.sql
```

В копию попадают вердикты, решения диспетчеров, заявки, пользователи и журнал действий — то есть всё, что накоплено эксплуатацией.

5.2. Возврат к исходному набору вердиктов

Стирает решения и заявки, возвращает 162 демонстрационных вердикта:

```
docker compose exec db psql -U concorde -c "TRUNCATE verdict CASCADE;"
docker compose exec app python -m scripts.06_seed_db
```

На сервере то же самое одной командой с подтверждением:

```
deploy/remote.sh reset
```

5.3. Обновление версии

Локально:

```
git pull
docker compose up -d --build
```

На сервере — повторное развёртывание, оно же обновление:

```
deploy/remote.sh deploy
```

Данные при этом сохраняются. Если менялся набор вердиктов, после обновления нужен раздел 5.2 — загрузка идемпотентна и сама себя не перезапишет.

6. Настройки окружения

6.1. Переменные приложения

Переменная	По умолчанию	Назначение
APP_PORT	8080	порт, на котором доступен интерфейс
BIND_ADDR	127.0.0.1	адрес привязки; 0.0.0.0 открывает порт в сеть
POSTGRES_PASSWORD	concorde	пароль базы; на сервере создаётся случайным
SESSION_SECRET	демонстрационный	секрет подписи сессий
API_TOKEN	пусто	если задан, API требует заголовок X-API-Token

Токен API по умолчанию пуст намеренно: проверяющий не должен наткнуться на 401 при осмотре интерфейсов обмена. Токен обязан быть из символов ASCII — значения заголовков HTTP передаются в latin-1, и кириллический токен до сервера не дойдёт неискажённым. Приложение проверяет это при старте и отказывается запускаться с внятным сообщением.

6.2. Переменные скрипта развёртывания

Переменная	По умолчанию	Назначение
CONCORDE_HOST	—	user@host, обязательна
CONCORDE_SSH_PORT	22	порт SSH
CONCORDE_DIR	concorde	каталог на сервере
CONCORDE_APP_PORT	8080	порт приложения на сервере
CONCORDE_BIND	127.0.0.1	адрес привязки на сервере
CONCORDE_LOCAL_PORT	равен порту стенда	локальный порт туннеля
CONCORDE_SSH_OPTS	—	ключи ssh, например -i ~/.ssh/stand

7. Если что-то пошло не так

Симптом	Причина	Что делать
Пароль SSH спрашивают много раз	старая версия скрипта открывала своё соединение на каждую операцию	обновиться; в текущей версии соединение одно на запуск, а <code>deploy/remote.sh keys</code> убирает вопрос совсем
Bind for 0.0.0.0:8080 failed: port is already allocated	порт занят другим сервисом	<code>APP_PORT=8081 docker compose up</code>
Приложение висит на «Ожидание PostgreSQL»	пароль в <code>.env</code> не совпадает с паролем в томе базы	либо вернуть прежний <code>.env</code> , либо пересоздать том: <code>docker compose down -v</code>
Стенд поднялся, но снаружи недоступен	так и задумано: привязка к <code>127.0.0.1</code>	туннель (раздел 4.1) или обратный прокси (4.2)
Порт открыт <code>0.0.0.0</code> , но извне молчит	межсетевой экран или security group	откройте входящий TCP на порт стенда
Страница загрузилась, но пустая	обратный прокси без проброса веб-сокетов	добавьте заголовки <code>Upgrade</code> и <code>Connection</code> (раздел 4.2)
После пересчёта вердиктов на стенде старые данные	загрузка идемпотентна и пропускает непустую таблицу	раздел 5.2
Недостаточно прав на странице настроек	раздел только для диспетчера ОДС	войдите под <code>ods</code>
неверный одноразовый код	код устарел или разошлись часы	ждите смены кода, проверьте время на устройстве
Сборка висит на <code>load metadata</code>	недоступен Docker Hub	проверьте сеть и состояние Docker Desktop, при необходимости перезапустите демон

7.1. Куда смотреть в первую очередь

```
curl http://localhost:8080/api/health # жив ли сервис и видит ли базу
docker compose ps                    # состояние контейнеров
docker compose logs --tail 50 app     # последние строки журнала
```

Ответ `{"status":"ok","database":true}` означает, что работают и приложение, и оперативный контур.